

In []: !ju

Walmart Case Study

SUHEL MANDAL

1. Importing Libraries, Dataset and basic checks

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import norm

import warnings
warnings.filterwarnings('ignore')

# style for all plots
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 5)
```

```
In [5]: from google.colab import files
uploaded = files.upload()
```

no files selected

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving walmart_data.csv to walmart_data.csv

```
In [6]: df_walmart = pd.read_csv('/content/walmart_data.csv')
```

```
In [145... df_walmart.head()
```

Out [145...

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_City
0	1000001	P00069042	F	0-17	10	A	
1	1000001	P00248942	F	0-17	10	A	
2	1000001	P00087842	F	0-17	10	A	
3	1000001	P00085442	F	0-17	10	A	
4	1000002	P00285442	M	55+	16	C	

2. Data Exploration

In [101...

```
print('\nShape of the Dataset (rows,columns):',df_walmart.shape)

print('\n' + '=' * 50)

print('\nColumns Names: ')
print(df_walmart.columns.tolist())

print('\n' + '=' * 50)

print('\nData Types of each columns: ')
print(df_walmart.dtypes)

print('\n' + '=' * 50)

print('\nMissing values in each columns: ')
print(df_walmart.isnull().sum())

print('\n' + '=' * 50)
```

Shape of the Dataset (rows,columns): (550068, 11)

Columns Names:

```
['User_ID', 'Product_ID', 'Gender', 'Age', 'Occupation', 'City_Category', 'Stay_In_Current_City_Years', 'Marital_Status', 'Product_Category', 'Purchase', 'Purchase_Clipped']
```

Data Types of each columns:

User_ID	int64
Product_ID	category
Gender	category
Age	category
Occupation	int64
City_Category	category
Stay_In_Current_City_Years	category
Marital_Status	int64
Product_Category	int64
Purchase	int64
Purchase_Clipped	int64
dtype:	object

Missing values in each columns:

User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category	0
Purchase	0
Purchase_Clipped	0
dtype:	int64

```
In [7]: print('\nDescriptive statistics for the numerical columns of the Dataset')
df_walmart.describe()
```

Descriptive statistics for the numerical columns of the Dataset

Out[7]:

	User_ID	Occupation	Marital_Status	Product_Category	
count	5.500680e+05	550068.000000	550068.000000	550068.000000	55
mean	1.003029e+06	8.076707	0.409653	5.404270	
std	1.727592e+03	6.522660	0.491770	3.936211	
min	1.000001e+06	0.000000	0.000000	1.000000	
25%	1.001516e+06	2.000000	0.000000	1.000000	
50%	1.003077e+06	7.000000	0.000000	5.000000	
75%	1.004478e+06	14.000000	1.000000	8.000000	
max	1.006040e+06	20.000000	1.000000	20.000000	

3. Data types and categorical conversion

```
In [100... cat_cols = df_walmart.select_dtypes(include=['object']).columns

for col in cat_cols:
    df_walmart[col] = df_walmart[col].astype('category')

print("\nCategorical columns converted to 'category' dtype")

df_walmart.dtypes
```

Categorical columns converted to 'category' dtype

Out [100...

0

User_ID	int64
Product_ID	category
Gender	category
Age	category
Occupation	int64
City_Category	category
Stay_In_Current_City_Years	category
Marital_Status	int64
Product_Category	int64
Purchase	int64
Purchase_Clipped	int64

dtype: object

4. Non-graphical analysis (value counts, unique)

In [102...

```

print('\nGender Value counts:')
print('\n' + '-' * 45)
print(df_walmart['Gender'].value_counts())

print('\n' + '=' * 50)

print('\nAge Group Value counts:')
print('\n' + '-' * 45)
print(df_walmart['Age'].value_counts())

print('\n' + '=' * 50)

print('\nCity Value counts:')
print('\n' + '-' * 45)
print(df_walmart['City_Category'].value_counts())

print('\n' + '=' * 50)

print('\nMarital Status Value counts:')
print('\n' + '-' * 45)
print(df_walmart['Marital_Status'].value_counts())

print('\n' + '=' * 50)

print('\nProduct Category Value counts:')

```

```

print('\n' + '-' * 45)
print(df_walmart['Product_Category'].value_counts())

print('\n' + '=' * 50)

print('\nNumber of unique User_IDs:',df_walmart['User_ID'].nunique())
print('\nNumber of unique Product_IDs:',df_walmart['Product_ID'].nunique())

print('\n' + '=' * 50)

```

Gender Value counts:

```

-----
Gender
M      414259
F      135809
Name: count, dtype: int64

```

Age Group Value counts:

```

-----
Age
26-35      219587
36-45      110013
18-25       99660
46-50       45701
51-55       38501
55+         21504
0-17        15102
Name: count, dtype: int64

```

City Value counts:

```

-----
City_Category
B      231173
C      171175
A      147720
Name: count, dtype: int64

```

Marital Status Value counts:

```

-----
Marital_Status
0      324731
1      225337
Name: count, dtype: int64

```

Product Category Value counts:

```
-----  
Product_Category  
5      150933  
1      140378  
8      113925  
11     24287  
2      23864  
6      20466  
3      20213  
4      11753  
16      9828  
15      6290  
13      5549  
10      5125  
12      3947  
7       3721  
18      3125  
20      2550  
19      1603  
14      1523  
17       578  
9        410  
Name: count, dtype: int64
```

```
=====
```

Number of unique User_IDs: 5891

Number of unique Product_IDs: 3631

```
=====
```

5. Univariate Plots

```
In [37]: fig, axes = plt.subplots(3,2, figsize = (15,10))  
  
# Histogram for Purchase Data  
axes[0,0].hist(df_walmart['Purchase'], bins = 50, color = '#C39BD3')  
axes[0,0].set_title('Distribution of Purchase Amount')  
axes[0,0].set_xlabel('Purchase Amount')  
axes[0,0].set_ylabel('Frequency')  
  
# Countplot for Age  
  
sns.countplot(x = 'Age', data = df_walmart, ax = axes[0,1], palette  
axes[0,1].set_title('Count of Customers by Age Group')  
axes[0,1].set_xlabel('Age Group')  
axes[0,1].set_ylabel('Count')  
  
# Countplot for Gender  
  
sns.countplot(x = 'Gender', data = df_walmart, ax = axes[1,0], pale
```

```

axes[1,0].set_title('Count of Customers by Gender')
axes[1,0].set_xlabel('Gender')
axes[1,0].set_ylabel('Count')

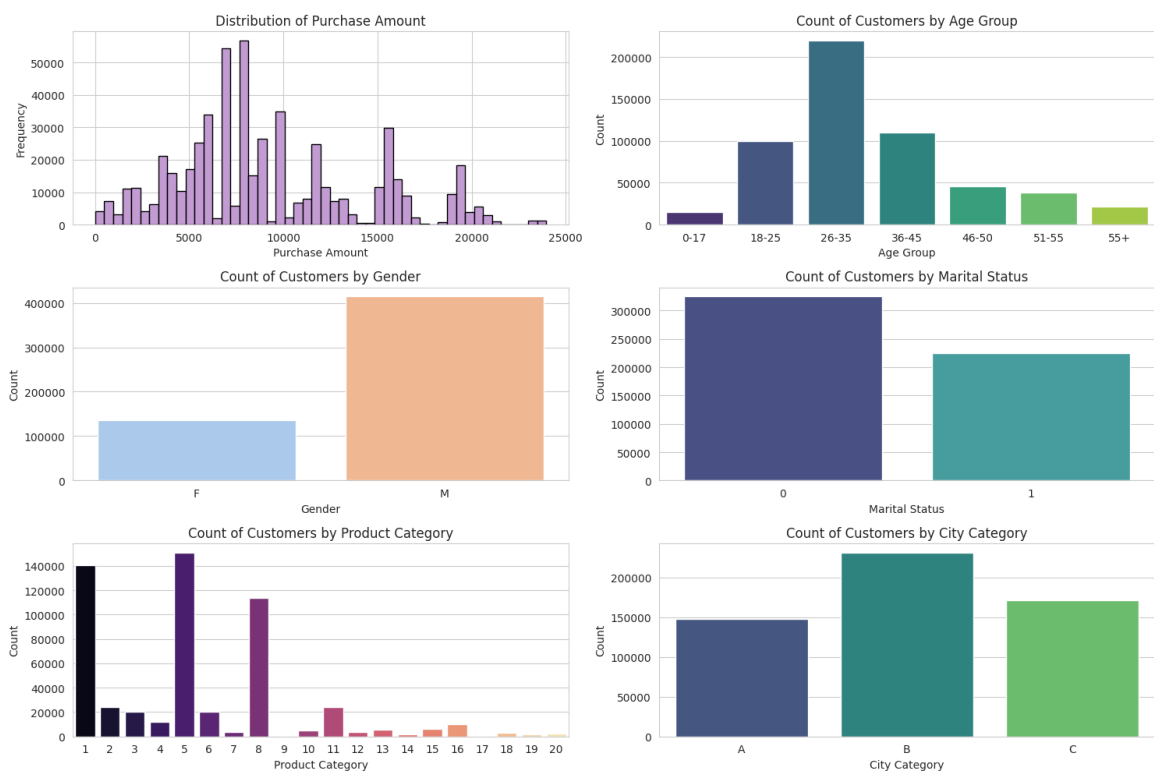
# Countplot of Marital Status
sns.countplot(x = 'Marital_Status', data = df_walmart, ax = axes[1,
axes[1,1].set_title('Count of Customers by Marital Status')
axes[1,1].set_xlabel('Marital Status')
axes[1,1].set_ylabel('Count')

# Countplot of Product category
sns.countplot(x = 'Product_Category', data = df_walmart, ax = axes[
axes[2,0].set_title('Count of Customers by Product Category')
axes[2,0].set_xlabel('Product Category')
axes[2,0].set_ylabel('Count')

# Countplot of City category
sns.countplot(x = 'City_Category', data = df_walmart, ax = axes[2,1
axes[2,1].set_title('Count of Customers by City Category')
axes[2,1].set_xlabel('City Category')
axes[2,1].set_ylabel('Count')

plt.tight_layout()
plt.show()

```



```

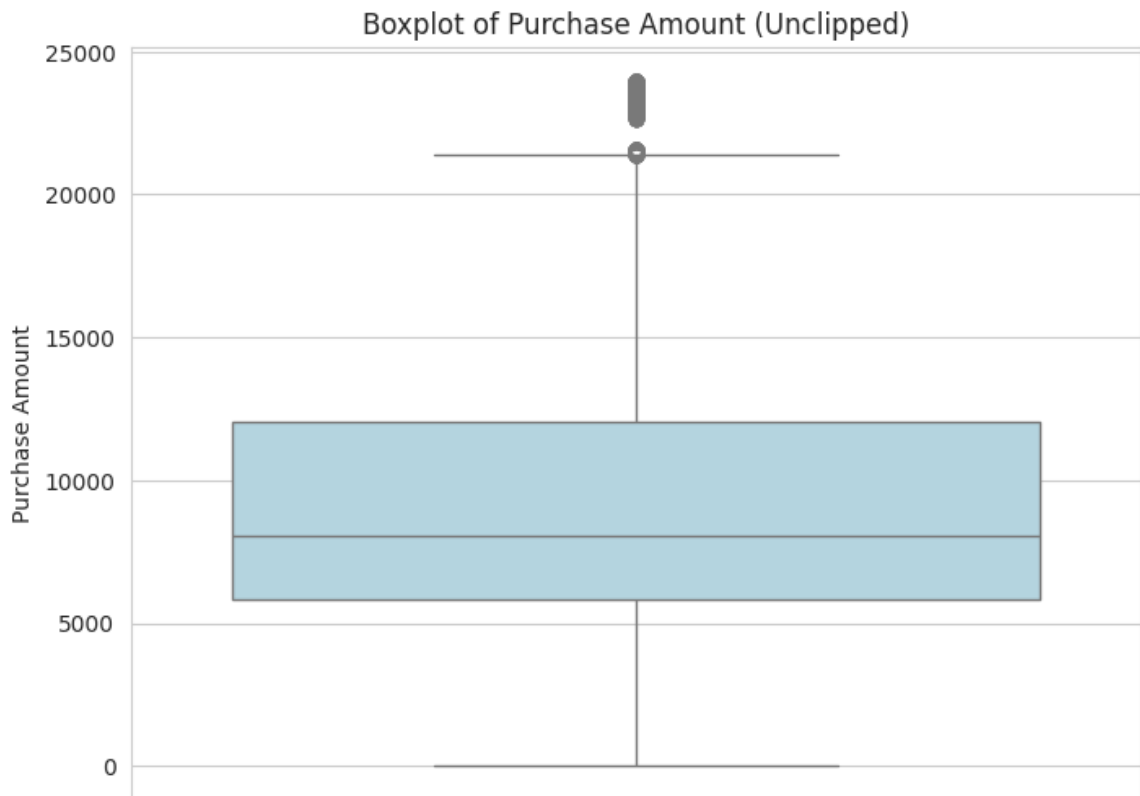
In [11]: # Boxplot of Purchase to see outliers

plt.figure(figsize = (8,6))
sns.boxplot(y = 'Purchase', data = df_walmart, color = 'lightblue')
plt.title('Boxplot of Purchase Amount (Unclipped)')
plt.ylabel('Purchase Amount')

```



```
plt.show()
```



6. Outlier detection and clipping

```
In [180... # Detect outliers and clip Purchase between 5% and 95%
Q5 = df_walmart['Purchase'].quantile(0.05)
Q95 = df_walmart['Purchase'].quantile(0.95)

print('5th Percentile:',Q5)
print('95th Percentile:',Q95)

df_walmart['Purchase_Clipped'] = np.clip(df_walmart['Purchase'],Q5,

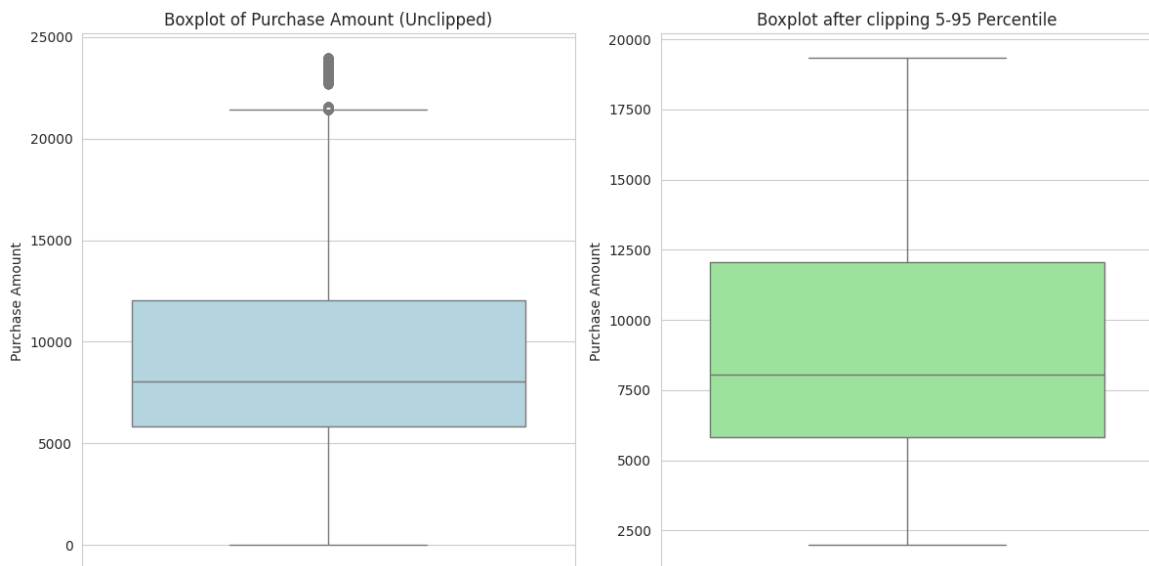
fig,axes = plt.subplots(1,2,figsize = (12,6)) # Changed to 1 row, 2

sns.boxplot(y = 'Purchase', data = df_walmart, ax=axes[0],color =
axes[0].set_title('Boxplot of Purchase Amount (Unclipped)')
axes[0].set_ylabel('Purchase Amount')

sns.boxplot(y = 'Purchase_Clipped', data = df_walmart, ax=axes[1],
axes[1].set_title('Boxplot after clipping 5-95 Percentile')
axes[1].set_ylabel('Purchase Amount')

plt.tight_layout()
plt.show()
```

```
5th Percentile: 1984.0
95th Percentile: 19336.0
```



7. Bivariate Analysis

```
In [57]: # Split by gender and compute average purchase
male_df = df_walmart[df_walmart["Gender"]=="M"]
female_df = df_walmart[df_walmart["Gender"]=="F"]

print('\n' + '=' * 150)
print('Male Sample Size:', len(male_df))
print('Female Sample Size:', len(female_df))
print('\n' + '-' * 50)

print(f"Average purchase by Male customers: {male_df['Purchase'].mean()}")
print(f"Average purchase by Female customers: {female_df['Purchase'].mean()}")
print('\n' + '-' * 50)

married_men = df_walmart[(df_walmart["Gender"] == "M") & (df_walmart["Married"] == True)]
married_women = df_walmart[(df_walmart["Gender"] == "F") & (df_walmart["Married"] == True)]

unmarried_men = df_walmart[(df_walmart["Gender"] == "M") & (df_walmart["Married"] == False)]
unmarried_women = df_walmart[(df_walmart["Gender"] == "F") & (df_walmart["Married"] == False)]

print(f"Average purchase by Married Male customers: {married_men['Purchase'].mean()}")
print(f"Average purchase by Married Women customers: {married_women['Purchase'].mean()}")
print('\n' + '-' * 50)

print(f"Average purchase by Un-Married Male customers: {unmarried_men['Purchase'].mean()}")
print(f"Average purchase by Un-Married Women customers: {unmarried_women['Purchase'].mean()}")
print('\n' + '=' * 150)

fig, axes = plt.subplots(4,2,figsize = (14,18))

sns.boxplot(x = 'Gender', y = 'Purchase', data = df_walmart, ax = axes[0,0])
axes[0,0].set_title('Boxplot of Purchase by Gender(Unclipped-Data)')
axes[0,0].set_ylabel('Purchase Amount')

sns.boxplot(x = 'Age', y = 'Purchase', data = df_walmart, ax = axes[0,1])
axes[0,1].set_title('Boxplot of Purchase by Age')
```

```

axes[0,1].set_ylabel('Purchase Amount')

sns.boxplot(x = 'Occupation', y = 'Purchase', data = df_walmart, ax
axes[1,0].set_title('Boxplot of Purchase by Occupation')
axes[1,0].set_ylabel('Purchase Amount')

sns.boxplot(x = 'City_Category', y = 'Purchase', data = df_walmart,
axes[1,1].set_title('Boxplot of Purchase by City Category')
axes[1,1].set_ylabel('Purchase Amount')

sns.boxplot(x = 'Product_Category', y = 'Purchase', data = df_walma
axes[2,0].set_title('Boxplot of Purchase by Product Category')
axes[2,0].set_ylabel('Purchase Amount')

sns.boxplot(x = 'Marital_Status', y = 'Purchase', data = df_walmart
axes[2,1].set_title('Boxplot of Purchase by Marital Status')
axes[2,1].set_ylabel('Purchase Amount')

sns.boxplot(x = 'Age', y = 'Purchase', data = df_walmart, hue = "Ma
axes[3,0].set_title('Boxplot of Purchase by Age & Marital Status')
axes[3,0].set_ylabel('Purchase Amount')

sns.boxplot(x = 'Product_Category', y = 'Purchase', data = df_walma
axes[3,1].set_title('Boxplot of Purchase by Product Category & Mari
axes[3,1].set_ylabel('Purchase Amount')

plt.tight_layout()
plt.show()

```

```

=====
=====
=====

```

Male Sample Size: 414259
Female Sample Size: 135809

Average purchase by Male customers: 9437.5260
Average purchase by Female customers: 8734.5658

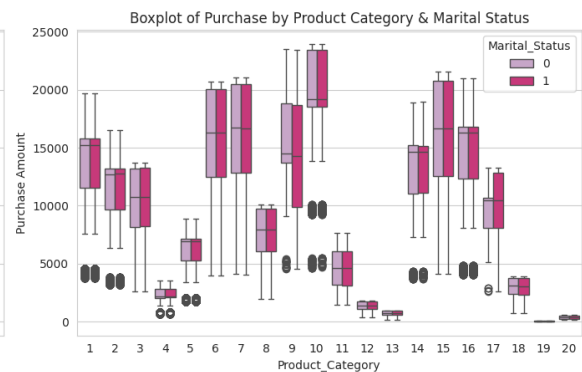
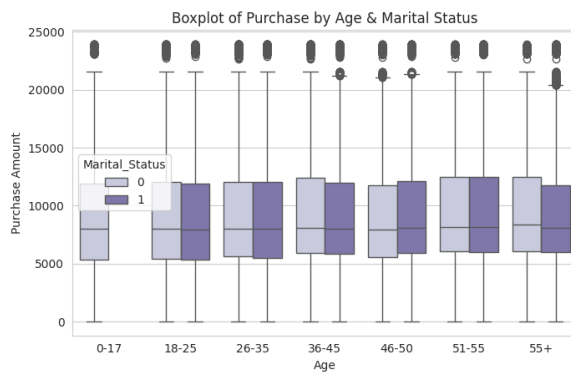
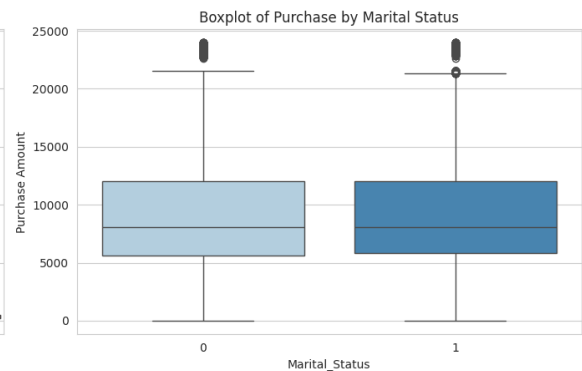
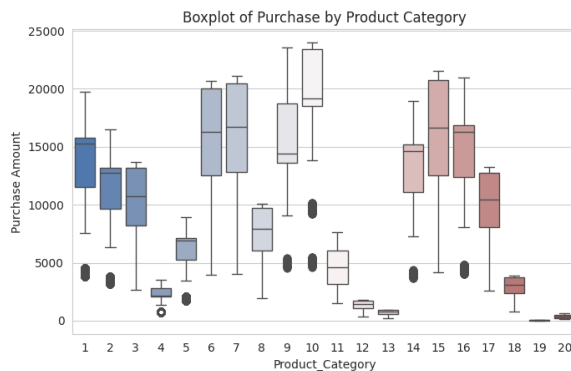
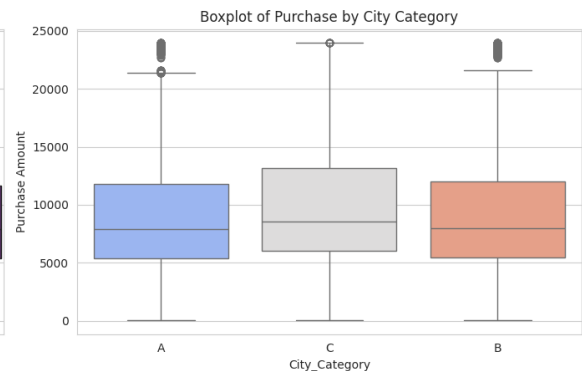
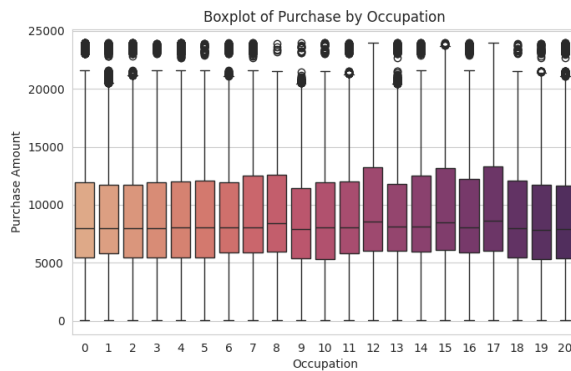
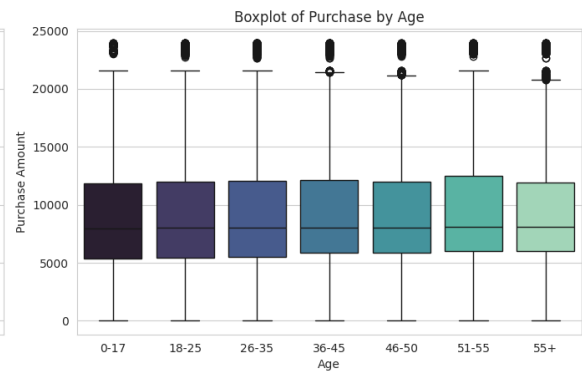
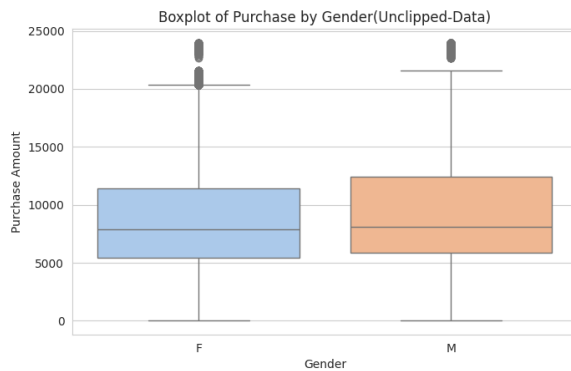
Average purchase by Married Male customers: 9413.8176
Average purchase by Married Women customers: 8810.2498

Average purchase by Un-Married Male customers: 9453.7567
Average purchase by Un-Married Women customers: 8679.8458

```

=====
=====
=====

```

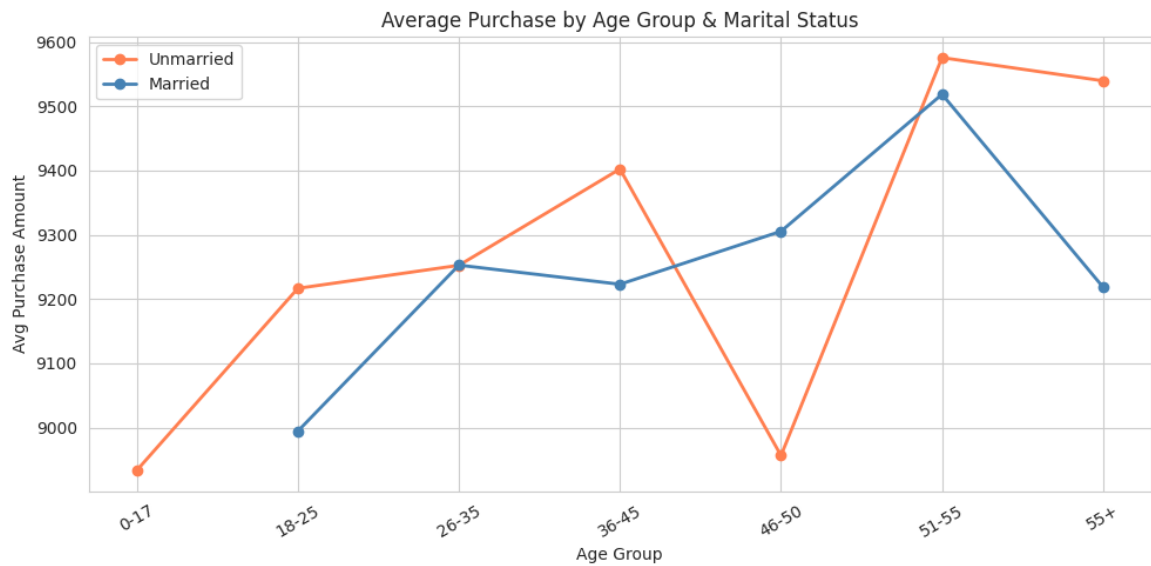


```
In [51]: # Line plot: Average spend by Age and Marital Status
age_order = ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
df_walmart['Age'] = pd.Categorical(df_walmart['Age'], categories=age_order)

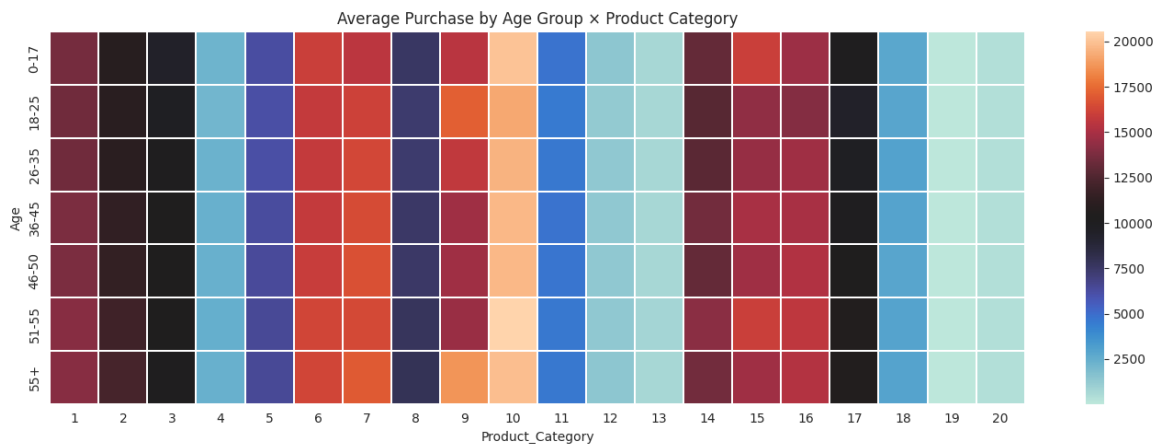
avg_age_marital = df_walmart.groupby(['Age', 'Marital_Status'])['Purchase Amount'].mean()

plt.figure(figsize=(10, 5))
for ms, label, color in [(0, 'Unmarried', 'coral'), (1, 'Married', 'blue')]:
    subset = avg_age_marital[avg_age_marital['Marital_Status'] == ms]
    plt.plot(subset['Age'].astype(str), subset['Purchase Amount'],
             marker='o', label=label, color=color, linewidth=2)
plt.title("Average Purchase by Age Group & Marital Status")
plt.xlabel("Age Group")
plt.ylabel("Avg Purchase Amount")
```

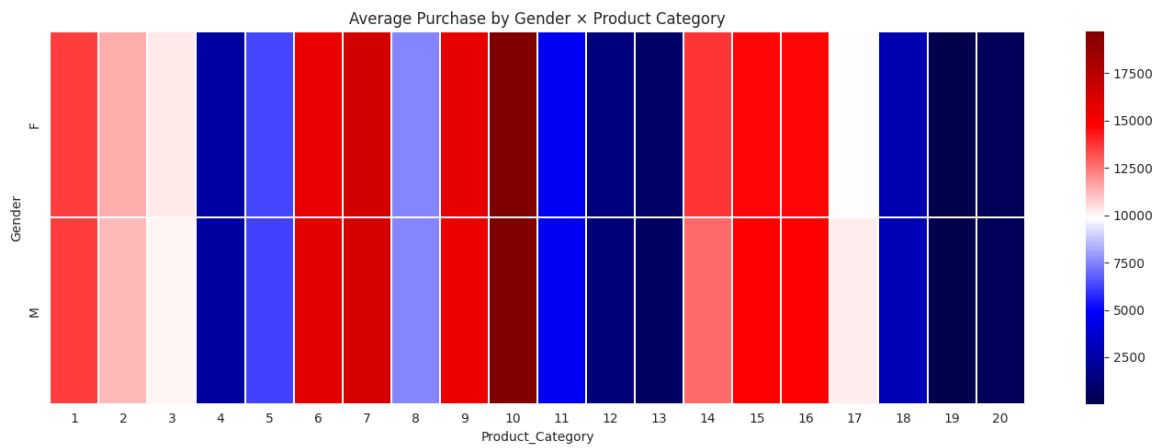
```
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



```
In [125... # Heatmap: Age vs Product Category (average spend)
age_product = df_walmart.groupby(['Age', 'Product_Category'])['Purc
plt.figure(figsize=(14, 5))
sns.heatmap(age_product, cmap='icefire', linewidths=0.3)
plt.title("Average Purchase by Age Group x Product Category")
plt.tight_layout()
plt.show()
```



```
In [119... # Heatmap: Gender vs Product Category (average spend)
Gender_product = df_walmart.groupby(['Gender', 'Product_Category'])
plt.figure(figsize=(14, 5))
sns.heatmap(Gender_product, cmap='seismic', linewidths=0.3)
plt.title("Average Purchase by Gender x Product Category")
plt.tight_layout()
plt.show()
```



In [88]: *# Heatmap of correlations (only numeric columns)*

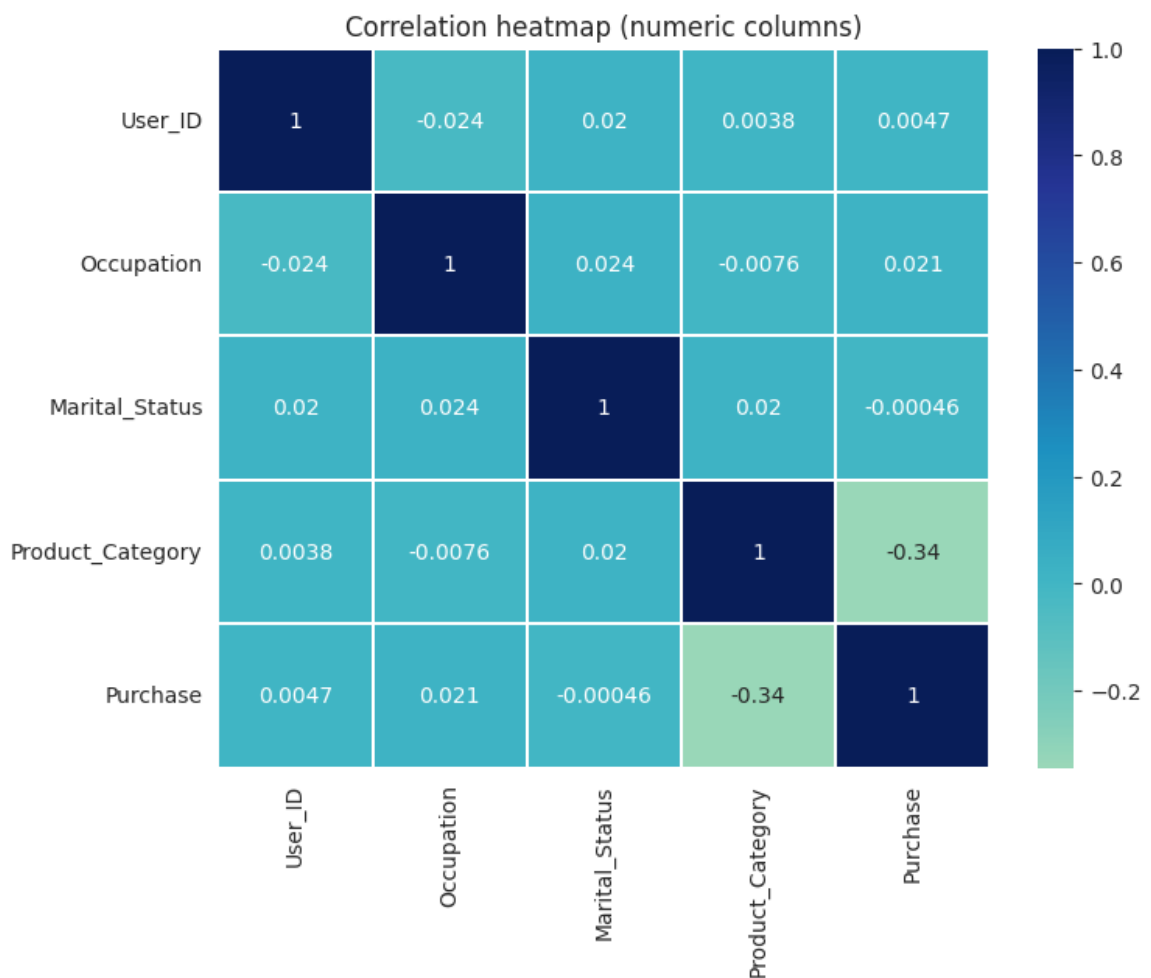
```
df_num = df_walmart.select_dtypes(include=[np.number])
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(df_num.corr(), annot=True, cmap='YlGnBu', center=0, li
```

```
plt.title("Correlation heatmap (numeric columns)")
```

```
plt.show()
```



8. CLT - Gender vs Purchase Amount

In [8]: **def** confidence_interval(data, confidence=0.95):

```

mean = np.mean(data)
std = np.std(data)
n = len(data)

z = norm.ppf(1 - (1 - confidence)/2)

margin_error = z * (std / np.sqrt(n))

lower = mean - margin_error
upper = mean + margin_error

return round(lower,2), round(upper,2)

```

```

In [9]: male_purchase_data = df_walmart[df_walmart['Gender'] == 'M']['Purch
female_purchase_data = df_walmart[df_walmart['Gender'] == 'F']['Pur

print('\n'+ '='*50)
print(f'Male Purchase data:\n{male_purchase_data}')
print('\n'+ '='*50)
print(f'Female Purchase data:\n{female_purchase_data}')
print('\n'+ '='*50)
print(f'Male CI: {confidence_interval(male_purchase_data)}')
print(f'Female CI: {confidence_interval(female_purchase_data)}')
print('\n'+ '='*50)

```

=====

Male Purchase data:

```
0      7969
1     15227
2     19215
3     15854
4     15686
```

...

```
414254      61
414255     121
414256     494
414257     473
414258     368
```

Name: Purchase, Length: 414259, dtype: int64

=====

Female Purchase data:

```
0      8370
1     15200
2      1422
3      1057
4     5378
```

...

```
135804      599
135805      371
135806      137
135807      365
135808      490
```

Name: Purchase, Length: 135809, dtype: int64

=====

Male CI: (np.float64(9422.02), np.float64(9453.03))

Female CI: (np.float64(8709.21), np.float64(8759.92))

=====

In [13]: *# Using Bootstrapping for Gender VS Purchase Amount*

```
for n in [300,3000,30000]:
    male_bootstrap_data = [np.mean(male_purchase_data.sample(n,replac
    female_bootstrap_data = [np.mean(female_purchase_data.sample(n,re

    bootstrap_ci_male = np.percentile(male_bootstrap_data,2.5),np.per
    print(f'The Confidence Interval for Male with {n} samples is: {bo
    bootstrap_ci_female = np.percentile(female_bootstrap_data,2.5),np
    print(f'The Confidence Interval for Female with {n} samples is: {

    overlap = not (bootstrap_ci_male[1] < bootstrap_ci_female[0] or b
    print(f'Do Male and Female CIs overlap for {n} samples? {overlap}

# The CI and Overlap plot
plt.figure(figsize=(10, 6))
sns.histplot(male_bootstrap_data, kde = True, color = 'blue', lab
sns.histplot(female_bootstrap_data, kde = True, color = 'red', la

# For 95% confidence interval for male data
```



```
plt.axvline(bootstrap_ci_male[0], color='blue', linestyle='--', l
plt.axvline(bootstrap_ci_male[1], color='blue', linestyle='--', l

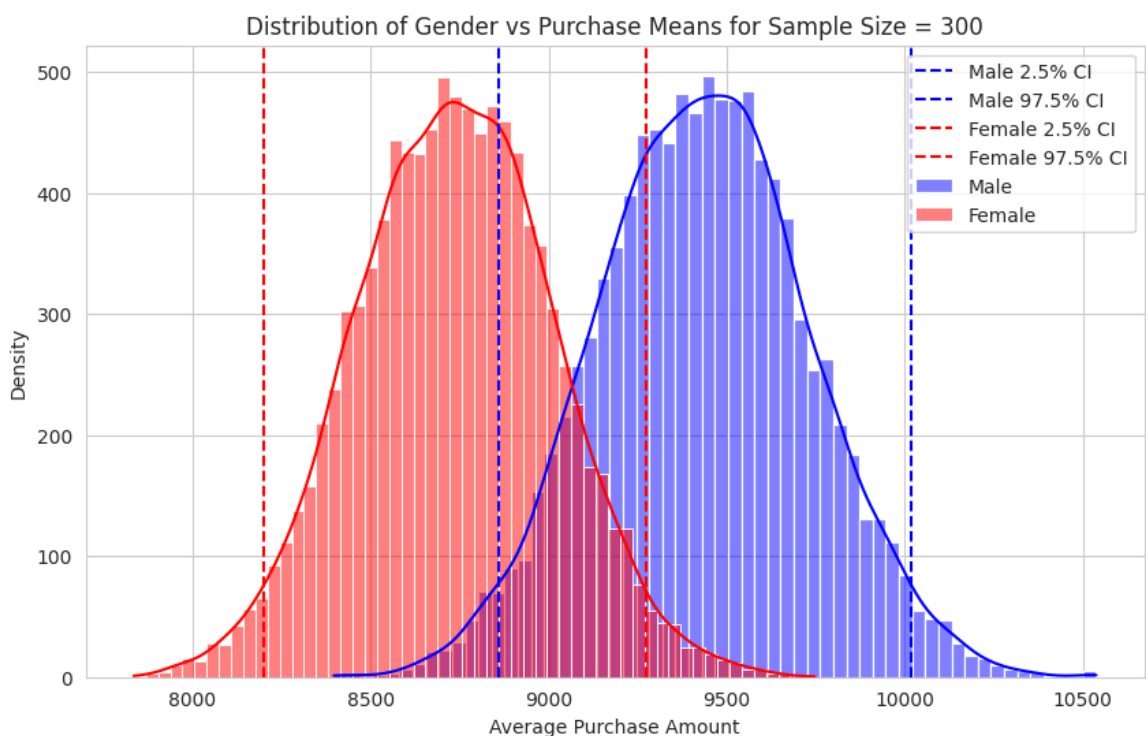
# fro 95% confidence interval for female data
plt.axvline(bootstrap_ci_female[0], color='red', linestyle='--',
plt.axvline(bootstrap_ci_female[1], color='red', linestyle='--',

plt.title(f'Distribution of Gender vs Purchase Means for Sample S
plt.xlabel('Average Purchase Amount')
plt.ylabel('Density')
plt.legend()
plt.show()
```

The Confidence Interval for Male with 300 samples is: (np.float64(8860.582666666667), np.float64(10019.017916666666))

The Confidence Interval for Female with 300 samples is: (np.float64(8201.34375), np.float64(9272.667916666667))

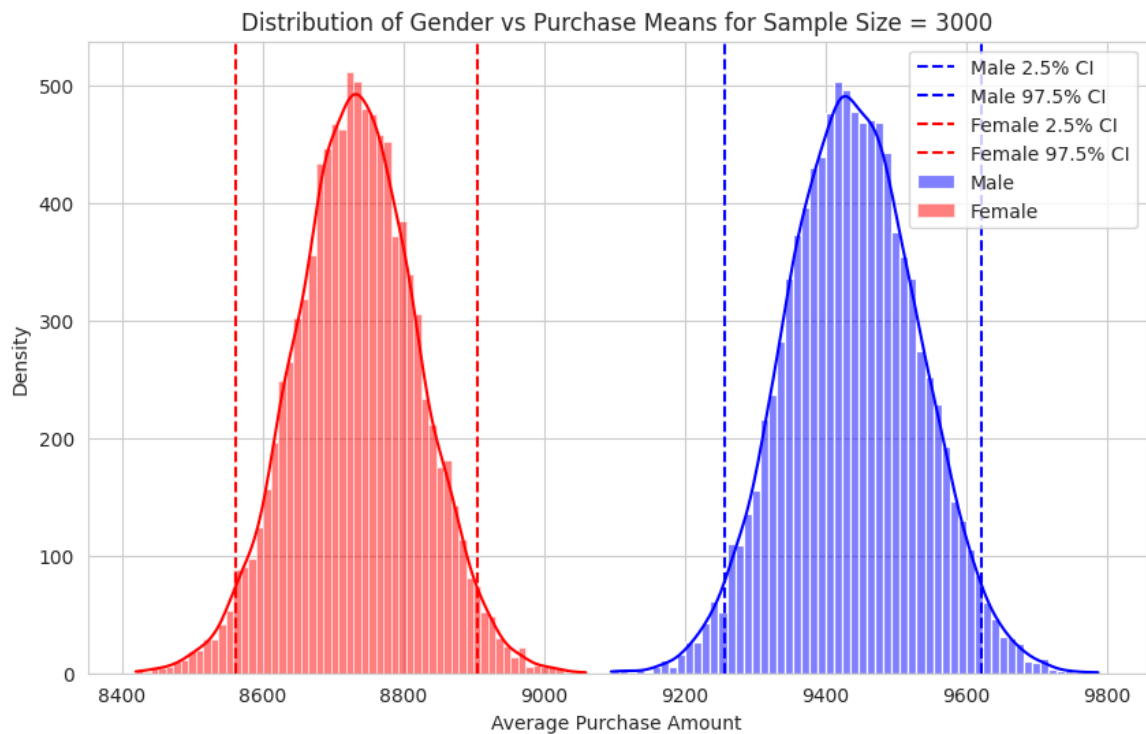
Do Male and Female CIs overlap for 300 samples? True



The Confidence Interval for Male with 3000 samples is: (np.float64(9257.593458333335), np.float64(9621.76435))

The Confidence Interval for Female with 3000 samples is: (np.float64(8561.242766666666), np.float64(8905.875283333333))

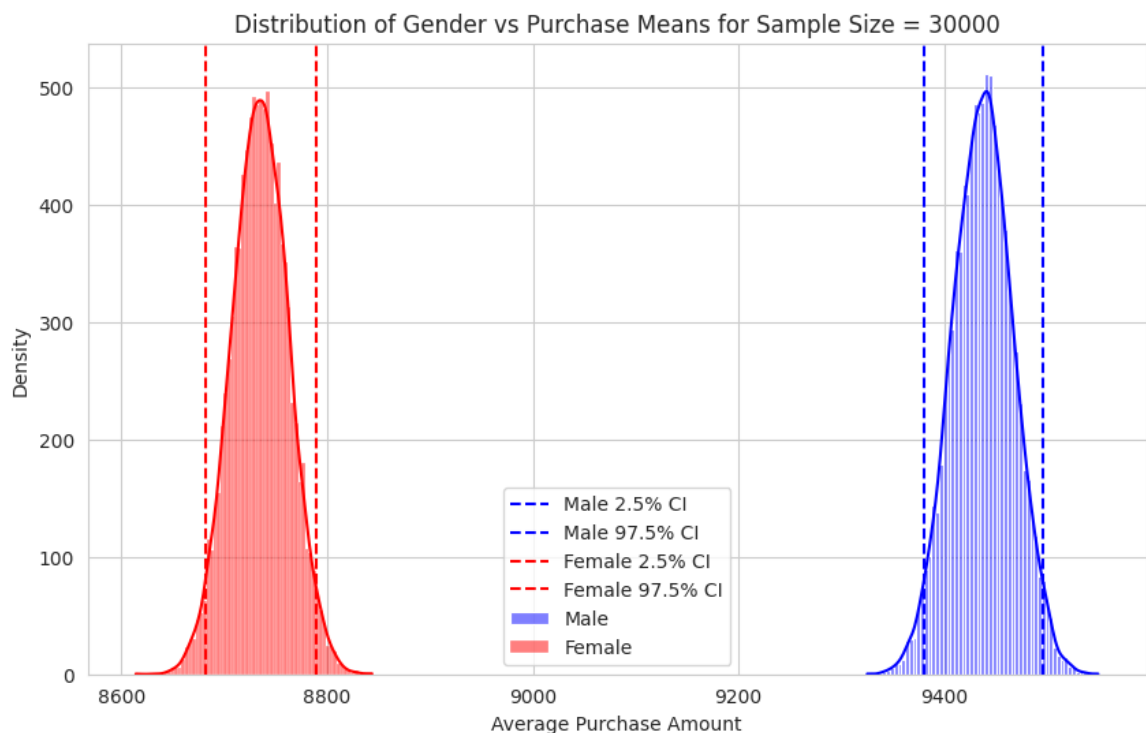
Do Male and Female CIs overlap for 3000 samples? False



The Confidence Interval for Male with 30000 samples is: (np.float64(9379.822745833333), np.float64(9495.842498333333))

The Confidence Interval for Female with 30000 samples is: (np.float64(8681.210098333333), np.float64(8789.434528333333))

Do Male and Female CIs overlap for 30000 samples? False



9. CLT - Marital Status vs Purchase Amount

```
In [11]: # Using the Confidence Interval Definition
```

```
married_data = df_walmart[df_walmart['Marital_Status'] == 1]['Purch
```

```

unmarried_data = df_walmart[df_walmart['Marital_Status']== 0]['Purc

print(f'Married_CI: {confidence_interval(married_data)}')
print(f'Unmarried_CI = {confidence_interval(unmarried_data)}')

```

Married_CI: (np.float64(9240.46), np.float64(9281.89))
 Unmarried_CI = (np.float64(9248.62), np.float64(9283.2))

In [27]: *# Using the Bootstrap Data*

```

for n in [300,3000,30000]:
    married_bootstrap_data = [np.mean(married_data.sample(n,replace =
    unmarried_bootstrap_data = [np.mean(unmarried_data.sample(n, repla

    bootstrap_ci_married = np.percentile(married_bootstrap_data,2.5),
    bootstrap_ci_unmarried = np.percentile(unmarried_bootstrap_data,2

    print(f'The Confidence Interval for Married with {n} samples is:
    print(f'The Confidence Interval for Unmarried with {n} samples is

    overlap = not (bootstrap_ci_married[1] < bootstrap_ci_unmarried[0]
    print(f'Do Male and Female CIs overlap for {n} samples? {overlap}

# The CI and Overlap plot
plt.figure(figsize=(10, 6))
sns.histplot(married_bootstrap_data, kde = True, color = 'blue',
sns.histplot(unmarried_bootstrap_data, kde = True, color = 'red',

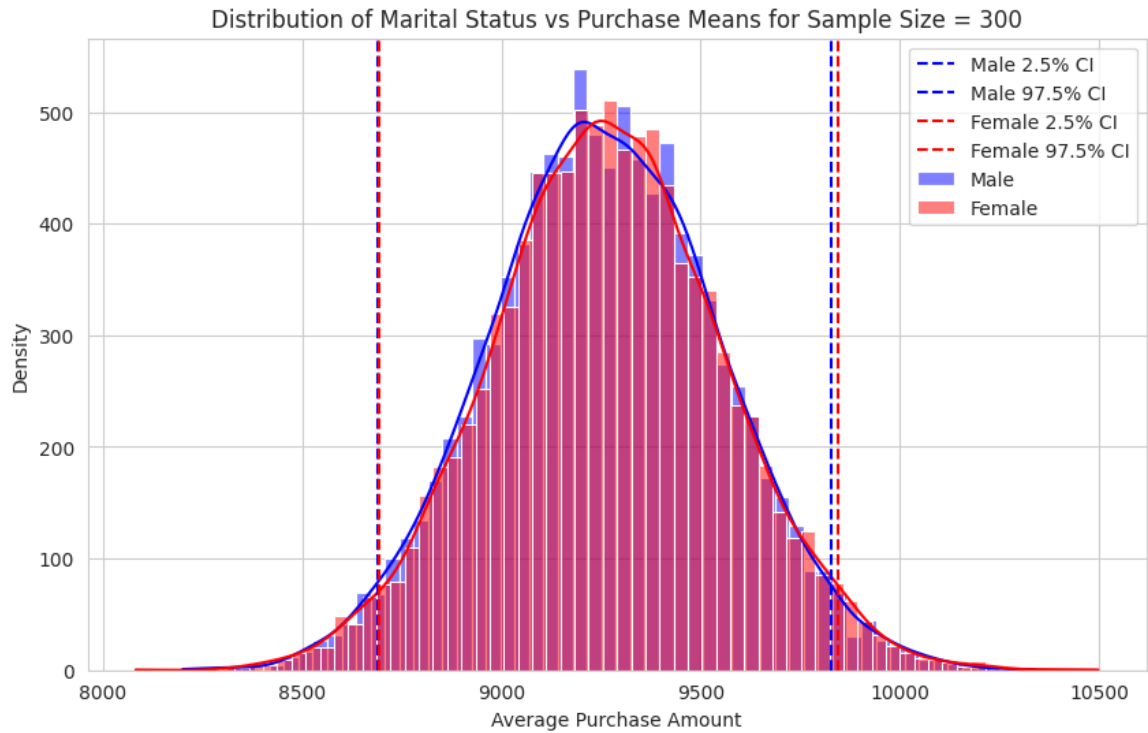
# For 95% confidence interval for male data
plt.axvline(bootstrap_ci_married[0], color='blue', linestyle='--'
plt.axvline(bootstrap_ci_married[1], color='blue', linestyle='--'

# fro 95% confidence interval for female data
plt.axvline(bootstrap_ci_unmarried[0], color='red', linestyle='--'
plt.axvline(bootstrap_ci_unmarried[1], color='red', linestyle='--

plt.title(f'Distribution of Marital Status vs Purchase Means for
plt.xlabel('Average Purchase Amount')
plt.ylabel('Density')
plt.legend()
plt.show()

```

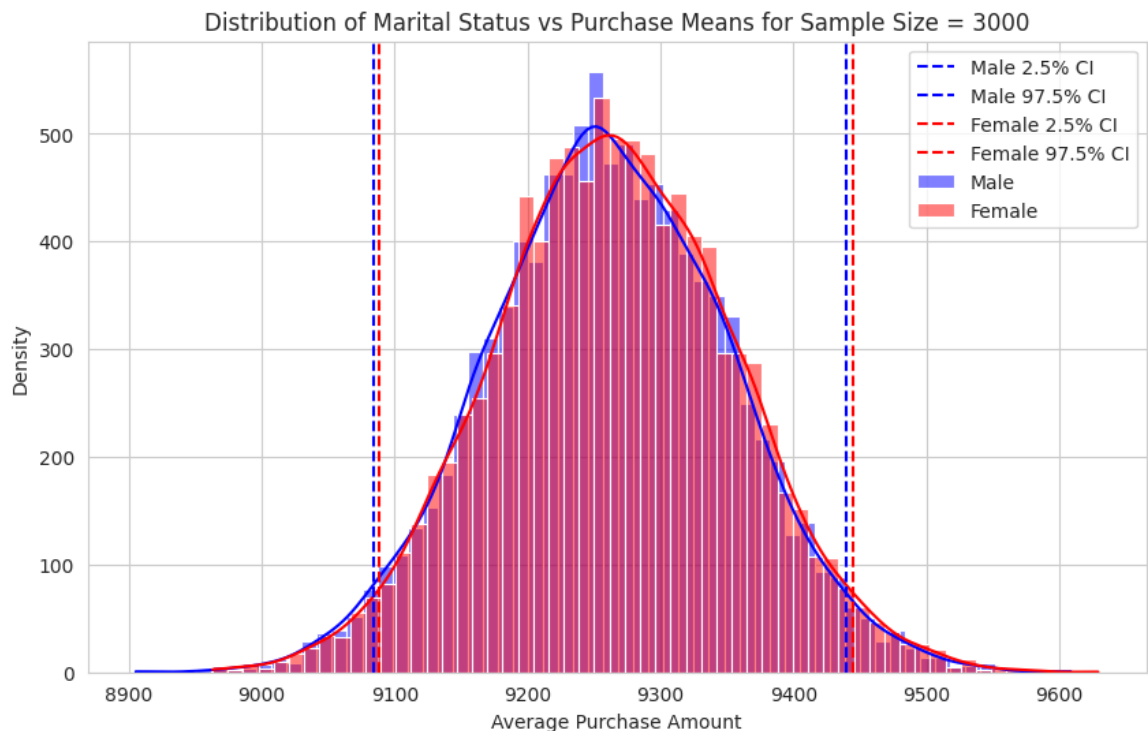
The Confidence Interval for Married with 300 samples is: (np.float64(8691.064333333332), np.float64(9828.62875))
 The Confidence Interval for Unmarried with 300 samples is: (np.float64(8691.533416666667), np.float64(9844.383083333332))
 Do Male and Female CIs overlap for 300 samples? True



The Confidence Interval for Married with 3000 samples is: (np.float64(9083.808141666666), np.float64(9439.226316666667))

The Confidence Interval for Unmarried with 3000 samples is: (np.float64(9087.869716666666), np.float64(9445.24005))

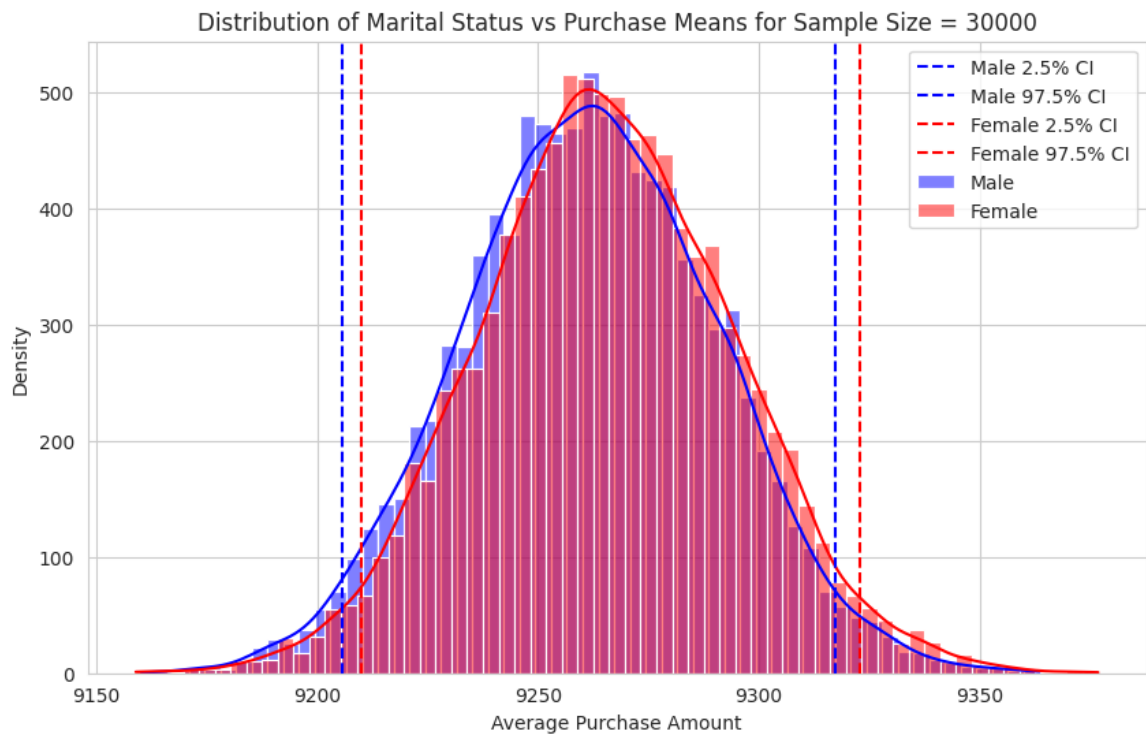
Do Male and Female CIs overlap for 3000 samples? True



The Confidence Interval for Married with 30000 samples is: (np.float64(9205.680725), np.float64(9317.361143333334))

The Confidence Interval for Unmarried with 30000 samples is: (np.float64(9209.9056375), np.float64(9323.027443333334))

Do Male and Female CIs overlap for 30000 samples? True



10. CLT - Age vs Purchase Amount

In [36]: *# Using the Confidence Interval Data*

```
age_data = df_walmart.groupby('Age')['Purchase'].apply(list)

for age, data in age_data.items():
    print(f'Age: {age}, CI: {confidence_interval(data)}')
```

```
Age: 0-17, CI: (np.float64(8851.95), np.float64(9014.98))
Age: 18-25, CI: (np.float64(9138.41), np.float64(9200.92))
Age: 26-35, CI: (np.float64(9231.73), np.float64(9273.65))
Age: 36-45, CI: (np.float64(9301.67), np.float64(9361.03))
Age: 46-50, CI: (np.float64(9163.09), np.float64(9254.17))
Age: 51-55, CI: (np.float64(9483.99), np.float64(9585.62))
Age: 55+, CI: (np.float64(9269.3), np.float64(9403.26))
```

In [31]: *# Show avg spend per age group*

```
avg_by_age = df_walmart.groupby('Age')['Purchase'].mean()
print("\nAverage Spend by Age Group:")
print(avg_by_age)

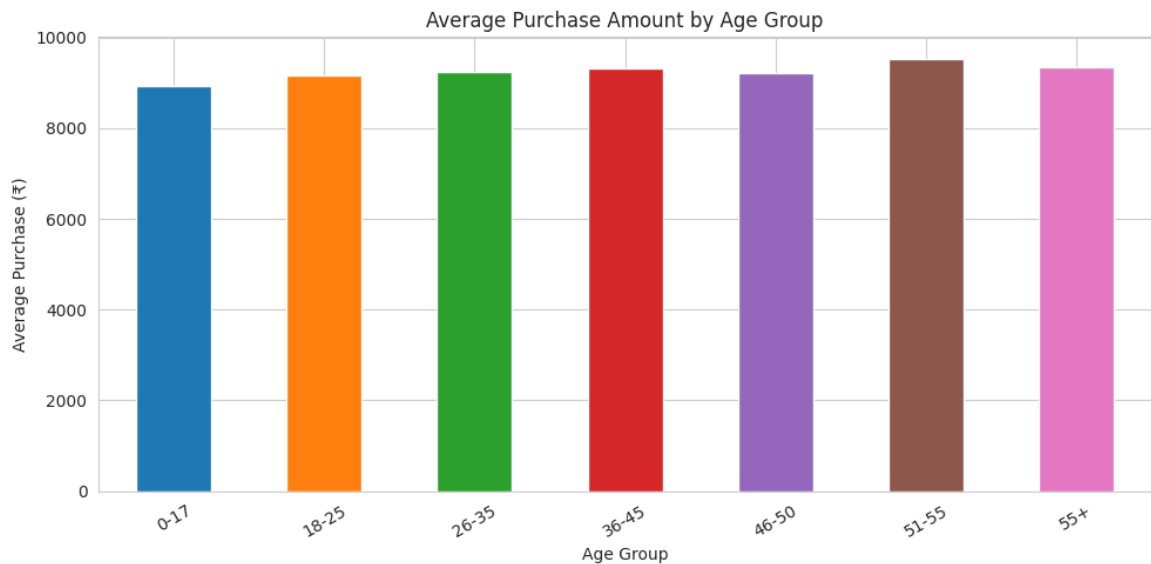
plt.figure(figsize=(10, 5))
avg_by_age.plot(kind='bar', color=age_colors, edgecolor='white')
plt.title("Average Purchase Amount by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Average Purchase (₹)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Average Spend by Age Group:

Age

0-17	8933.464640
18-25	9169.663606
26-35	9252.690633
36-45	9331.350695
46-50	9208.625697
51-55	9534.808031
55+	9336.280459

Name: Purchase, dtype: float64



```
In [44]: # Using Bootstrapping for Age Vs Purchase data

grouped_purchase_data = df_walmart.groupby('Age')['Purchase']

for n in [300, 3000, 30000]:
    age_bootstrap_data = {age_group_label: [np.mean(group_series.samp
    print(f"\n\nThe Confidence Intervals for {n} samples:")

    for age_group_label, bootstrap_data in age_bootstrap_data.items():
        bootstrap_ci = np.percentile(bootstrap_data, 2.5), np.percentile(
        print(f'\n\nThe Confidence Interval for {age_group_label} with {n
```

The Confidence Intervals for 300 samples:

The Confidence Interval for 0-17 with 300 samples is: (np.float64(8365.48375), np.float64(9522.449916666666))

The Confidence Interval for 18-25 with 300 samples is: (np.float64(8600.679750000001), np.float64(9742.752083333333))

The Confidence Interval for 26-35 with 300 samples is: (np.float64(8684.187), np.float64(9832.002666666665))

The Confidence Interval for 36-45 with 300 samples is: (np.float64(8766.248916666666), np.float64(9902.055))

The Confidence Interval for 46-50 with 300 samples is: (np.float64(8

674.843499999999), np.float64(9772.433166666666))

The Confidence Interval for 51–55 with 300 samples is: (np.float64(8966.472), np.float64(10109.762166666666))

The Confidence Interval for 55+ with 300 samples is: (np.float64(8776.533916666667), np.float64(9905.9735))

The Confidence Intervals for 3000 samples:

The Confidence Interval for 0–17 with 3000 samples is: (np.float64(8752.5317), np.float64(9118.606858333333))

The Confidence Interval for 18–25 with 3000 samples is: (np.float64(8990.4131), np.float64(9350.614266666667))

The Confidence Interval for 26–35 with 3000 samples is: (np.float64(9069.052258333333), np.float64(9432.577674999999))

The Confidence Interval for 36–45 with 3000 samples is: (np.float64(9151.70505), np.float64(9505.735758333334))

The Confidence Interval for 46–50 with 3000 samples is: (np.float64(9025.802508333332), np.float64(9386.183816666666))

The Confidence Interval for 51–55 with 3000 samples is: (np.float64(9351.024691666667), np.float64(9715.994491666666))

The Confidence Interval for 55+ with 3000 samples is: (np.float64(9153.280458333333), np.float64(9516.186599999999))

The Confidence Intervals for 30000 samples:

The Confidence Interval for 0–17 with 30000 samples is: (np.float64(8875.859827499999), np.float64(8990.517265833332))

The Confidence Interval for 18–25 with 30000 samples is: (np.float64(9111.622669166667), np.float64(9226.416495000001))

The Confidence Interval for 26–35 with 30000 samples is: (np.float64(9196.6478575), np.float64(9310.284010833333))

The Confidence Interval for 36–45 with 30000 samples is: (np.float64(9276.064676666667), np.float64(9387.8740575))

The Confidence Interval for 46–50 with 30000 samples is: (np.float64(9153.771598333333), np.float64(9264.840862500001))

The Confidence Interval for 51–55 with 30000 samples is: (np.float64(9477.8971575), np.float64(9592.439284166667))

The Confidence Interval for 55+ with 30000 samples is: (np.float64(9279.924049166666), np.float64(9392.287463333334))

Observations

Customer Demographics

The dataset skews heavily male — men account for roughly 75% of all transactions. The dominant age group is 26–35, followed by 18–25 and 36–45, meaning the core customer base is young-to-mid adults. Marital status is slightly skewed toward unmarried customers. City Category B has the highest transaction volume, followed by C and then A.

Purchase Distribution

The purchase amount distribution is right-skewed, with a long tail of high-value transactions. The boxplot before clipping shows significant outliers on the upper end. After clipping at the 5th and 95th percentiles, the distribution becomes much cleaner, suggesting a small set of very high-value purchases distort summary statistics.

Bivariate Patterns

Male customers consistently spend more on average than female customers across almost all product categories and age groups. Married vs. unmarried status shows very little difference in average spend, with confidence intervals overlapping even at large sample sizes ($n=30,000$) — indicating marital status is not a meaningful predictor of purchase amount. Product Category is the strongest differentiator of purchase amount, with certain categories commanding dramatically higher average spend.

CLT & Bootstrapping

For Gender: at all sample sizes (300, 3000, 30000), the male CI is consistently higher than the female CI, and by $n=3000$ and beyond, the CIs stop overlapping — confirming the gender spend difference is statistically real, not noise.

For Marital Status: CIs overlap at all sample sizes, meaning there is no statistically significant difference in spend between married and unmarried customers.

For Age: the 26–35 group shows the highest total spend volume; however, older groups (46–55+) may show higher average spend per transaction due to smaller sample sizes inflating the mean.

Insights

Target male 26–35 segment.

This cohort drives the largest share of revenue. Personalized offers and loyalty programs should prioritize this demographic.

Expand female customer acquisition

Women are under-represented (~25%) despite a statistically lower but non-trivial spend. Targeted campaigns could unlock significant revenue.

High-value product categories

Certain product categories command significantly higher average spend. Ensure these are prominently placed and well-stocked.

City B is the revenue hub

City Category B has the highest transaction count. Local promotions and inventory depth should reflect this concentration.

Handle outliers carefully

High-value outliers (~top 5%) inflate average spend metrics. Use clipped or median-based metrics for fairer segmentation analysis.

Recommendation

```
In [89]: recommendations = """
1. GENDER – TARGET FEMALE CUSTOMERS MORE:
    Males spend ~₹1,000+ more per transaction than females.
    The CIs do NOT overlap, confirming this is a real difference.
    Action: Create special Black Friday deals for women – on categories
    like beauty, apparel, and household goods – to increase their spend.

2. MARITAL STATUS – NOT A KEY DIFFERENTIATOR:
    Married and unmarried customers spend almost the same amount.
    The CIs overlap, so there is no strong spending difference.
    Action: Do NOT create separate promotions just based on marital status.
    Focus on other variables like gender and age instead.

3. AGE – FOCUS ON 26–45 YEAR OLDS:
    Age groups 26–35 and 36–45 spend the most on Black Friday.
    Action: Heavily promote premium products (electronics, appliances) to
    this group. They are the primary revenue-generating segment.

4. YOUNGER CUSTOMERS (18–25):
    They spend less but transact frequently.
    Action: Offer trendy, affordable products and flash deals to attract
    this group and build brand loyalty for the long term.

5. SENIOR CUSTOMERS (55+):
    They spend the least. They may need help navigating deals.
    Action: Provide simpler shopping experiences, gift guides, and
    dedicated customer support during Black Friday.

6. CITY CATEGORY:
    Analyze which city types drive the most revenue.
    Action: Run larger in-store events in cities that drive higher spend.

7. PRODUCT CATEGORIES:
    Some product categories drive much higher purchase amounts.
    Action: Make sure high-value product categories are well-stocked
    and prominently featured during Black Friday.
"""

print(recommendations)
```

1. GENDER – TARGET FEMALE CUSTOMERS MORE:

Males spend ~₹1,000+ more per transaction than females.

The CIs do NOT overlap, confirming this is a real difference.

Action: Create special Black Friday deals for women – on categories

like beauty, apparel, and household goods – to increase their spending.

2. MARITAL STATUS – NOT A KEY DIFFERENTIATOR:

Married and unmarried customers spend almost the same amount.

The CIs overlap, so there is no strong spending difference.

Action: Do NOT create separate promotions just based on marital status.

Focus on other variables like gender and age instead.

3. AGE – FOCUS ON 26–45 YEAR OLDS:

Age groups 26–35 and 36–45 spend the most on Black Friday.

Action: Heavily promote premium products (electronics, appliances) to

this group. They are the primary revenue-generating segment.

4. YOUNGER CUSTOMERS (18–25):

They spend less but transact frequently.

Action: Offer trendy, affordable products and flash deals to attract

this group and build brand loyalty for the long term.

5. SENIOR CUSTOMERS (55+):

They spend the least. They may need help navigating deals.

Action: Provide simpler shopping experiences, gift guides, and dedicated customer support during Black Friday.

6. CITY CATEGORY:

Analyze which city types drive the most revenue.

Action: Run larger in-store events in cities that drive higher spend.

7. PRODUCT CATEGORIES:

Some product categories drive much higher purchase amounts.

Action: Make sure high-value product categories are well-stocked and prominently featured during Black Friday.